

Self-Reflective Report

Object-Oriented Programming (BIT1123)

Student Name: Elmenoufy Mohamed Ezat Abdelhamid

Student ID: 202401010544

Program: Bachelor of Information Technology

Course: BIT1123 Object-Oriented Programming

Faculty: Faculty of Information Technology

Institution: City University Malaysia

GitHub Repository: Elmenoufy-Mohamed-Ezat-Abdelhamid202401010544.OOP

Date: August 2026

1. Introduction

This self-reflective report describes my learning journey throughout the ten weeks of practical tutorials for the BIT1123 Object-Oriented Programming course. The practical work was organized in a GitHub repository containing my tutorial exercises from Week 1 to Week 10.

At the beginning of the course, I had only a basic understanding of programming and Java. Throughout the tutorials, I gradually developed my knowledge of Java syntax, classes, objects, encapsulation, inheritance, abstraction, polymorphism, and GUI programming. I also improved my ability to organize my work using GitHub and to solve programming problems through practical exercises.

The tutorials helped me understand that programming is not only about writing code that works, but also about organizing code into clear and reusable structures. This report reflects on what I learned, the challenges I faced, how I improved, and my plans for further development.

2. Knowledge Gained from Each Tutorial

Week 1: Introduction to Java

In Week 1, I started with the basic structure of a Java program. I practiced creating simple Java programs such as HelloWorld and StudentGrade. I learned about the main() method, Java syntax, output using System.out.println(), variables, and basic program execution. This week helped me become more comfortable with the Java development environment.

Week 2: Classes, Objects, Constructors, and Methods

In Week 2, I worked with a Student class containing attributes such as name, age, and

GPA. I learned how to create a class, define attributes and methods, and create objects using a constructor. I also practiced methods such as displaying student information, studying, and taking an exam.

Weeks 3–4: Classes, Constructors, and Object Relationships

During Weeks 3 and 4, I continued working with object-oriented programming concepts through classes such as Person, Student, and Lecturer. I practiced constructors, attributes, methods, and relationships between different classes.

Week 5: Encapsulation

Week 5 was an important step in my understanding of Object-Oriented Programming. I worked with a Student class whose attributes such as student ID, name, CGPA, and programme were declared as private. I learned how getter and setter methods can be used to provide controlled access to private data. This helped me understand encapsulation.

Week 6: Inheritance

In Week 6, I learned more about inheritance by creating an Employee class and a Lecturer class. The Lecturer class extends the Employee class, allowing it to reuse attributes and methods from the parent class. I also practiced using the super keyword to call the parent class constructor.

Week 7: Abstract Classes and Method Overriding

Week 7 introduced me to abstract classes and method overriding. I worked with an abstract Appliance class and subclasses such as WashingMachine and Refrigerator. The Appliance class contained an abstract operate() method, while each subclass provided its own implementation.

Weeks 8–9: Object-Oriented Programming Practice

During Weeks 8 and 9, I continued practicing the concepts learned in previous tutorials. I worked with Java classes and developed my understanding of object-oriented design through practical programming tasks. These weeks helped me become more comfortable with creating classes, using objects, applying inheritance and overriding methods, and organizing Java programs.

Week 10: Java GUI and Event Handling

In Week 10, I worked on a Java GUI project called Programming Quiz Battle. The project used Java Swing components such as JFrame, JLabel, and JButton. I also implemented event handling using ActionListener. This showed me how Java can be used to create an interactive application rather than only console-based programs.

3. Challenges Encountered

Throughout the ten weeks, I faced several challenges while learning Java and Object-Oriented Programming.

Understanding Java Syntax: At the beginning, I sometimes made simple syntax mistakes such as missing semicolons, incorrect brackets, or incorrect variable declarations.

Understanding Classes and Objects: One of my first challenges was understanding the difference between a class and an object and how objects are created from classes.

Understanding Encapsulation: I needed time to understand why variables should sometimes be private instead of public. Working with getters and setters helped me understand controlled access to data.

Understanding Inheritance: I had to understand how a child class can reuse properties and methods from a parent class and how the super keyword works.

Understanding Abstraction: Abstract classes and abstract methods were initially difficult because they cannot always be used directly like normal classes and methods.

GitHub and Version Control: Using GitHub alongside learning Java was also challenging. I had to learn commands such as git add, git commit, and git push while keeping my weekly work organized.

4. How I Overcame the Challenges

I used several methods to improve my understanding and solve the problems I encountered.

First, I practiced the tutorial examples repeatedly. Rewriting the code helped me remember the syntax and understand how different parts of a Java program work together.

Second, I learned to read compiler and runtime error messages instead of simply trying random changes. This helped me identify the causes of programming errors more effectively.

I also studied the code examples carefully and changed small parts of the programs to see how the output would change. This was especially useful when learning inheritance, encapsulation, and abstract classes.

Another useful strategy was organizing my work into separate weekly folders in GitHub. The repository contains separate folders for the different tutorial weeks, making it easier to review my progress.

Finally, I used online learning resources and discussed difficult programming concepts with classmates when necessary.

5. Improvements in My Java Programming Skills

My Java programming skills improved significantly during the course.

At the beginning, I mainly focused on writing simple programs and getting them to run. By the end of the tutorials, I became more comfortable designing classes and connecting different parts of a program.

I improved my ability to:

- Create Java classes and objects.
- Use constructors and methods.

- Apply encapsulation using private variables and getters/setters.
- Use inheritance between classes.
- Override methods in subclasses.
- Work with abstract classes and abstract methods.
- Create simple graphical user interfaces using Java Swing.
- Handle user actions using ActionListener.
- Organize programming exercises using GitHub.
- Debug and correct programming errors.

6. My Understanding of Object-Oriented Programming

The course helped me understand the main ideas behind Object-Oriented Programming in a practical way.

Encapsulation: I learned that encapsulation means protecting the internal data of a class and providing controlled access through methods such as getters and setters.

Inheritance: I learned that inheritance allows one class to reuse features from another class. For example, my Lecturer class extends the Employee class.

Abstraction: I learned that abstract classes can provide a common structure while leaving some behavior to be implemented by subclasses. My Appliance example demonstrated this concept through the abstract operate() method.

Polymorphism and Method Overriding: Through subclass implementations, I learned that the same method can behave differently depending on the object or subclass that implements it.

Overall, these concepts changed the way I think about programming. Instead of thinking only about individual lines of code, I now think more about objects, their responsibilities, and how they interact with each other.

7. Future Learning Plans

After completing this course, I want to continue improving my programming skills.

1. Improving my Java skills by building more complete applications.
2. Learning data structures and algorithms to improve my problem-solving ability.
3. Learning database programming and connecting Java applications to databases using SQL and JDBC.
4. Improving GUI development skills and creating more interactive applications.
5. Learning advanced Object-Oriented Programming concepts and design patterns.
6. Improving my Git and GitHub skills, especially branching, pull requests, and collaborative development.
7. Building personal projects to apply the knowledge gained from this course.

I believe that practicing through real projects will help me move from understanding programming concepts to applying them independently.

8. Conclusion

The BIT1123 Object-Oriented Programming course was an important part of my development as an Information Technology student. During the ten weeks, I progressed from writing simple Java programs to creating classes, using encapsulation and inheritance, working with abstract classes, and developing a simple interactive GUI application.

The practical tutorials helped me understand that programming requires patience, practice, and problem-solving. The challenges I faced with Java syntax, object-oriented concepts, debugging, and GitHub gradually became easier as I practiced more.

Creating and organizing my tutorial work in GitHub also gave me experience with a professional development tool that I can continue using in future university projects and software development work.

Overall, this course gave me a stronger foundation in Java and Object-Oriented Programming. I believe the knowledge and skills I gained will be useful in my future studies and in developing more advanced software projects.

GitHub Repository:

<https://github.com/medoelmenofy6-bot/Elmenoufy-Mohamed-Ezat-Abdelhamid202401010544.OOP>